

SDP



Module 0: The Challenge



Building FanHub in 8 Hours

"Prove that AI-assisted development is worth the hype by building something real. Fast."

Before We Start

WHAT YOU'LL NEED



GitHub account with Copilot access

Individual, Business, or Enterprise — any tier works



Recent version of VS Code + GitHub Copilot extension

Enable agent mode in settings if not already on



Git + local stack (node/dotnet/java/go), access to public packages

For running the FanHub starter app locally



FanHub starter repo cloned

Link in the Reference Material slide at the end

DIFFICULTY



Moderate

HANDS-ON



Build as you learn

PAYOFF



More every hour

RELEVANCE



If you use Copilot today

WHAT WE'LL COVER

00 Orientation — the challenge, the team, the mindset shift

01 Instructions — teaching Copilot your repo's conventions

02 Agent Plan Mode — structured thinking before doing

03 Custom Prompts — turning repeatable work into reusable assets

04 Agent Skills — encoding domain knowledge Copilot can load

05 MCP Servers — connecting agents to real tools and data

06 Custom Agents — pulling it all together autonomously

Meet The Team



Sarah

Senior Dev - 15 years

— *only skeptical* —

"I've outlasted three 'revolutionary' tools this decade. I'll believe it when my PR review time drops."



Marcus

DevOps Dev - 5 years

— *chronically distracted* —

"Sorry — I was half-reading a Hacker News thread about this. What were you saying?"



David

Staff Engineer - 20 years

— *pleasantly pedantic* —

"I'm not afraid of being replaced. I'm afraid of being the last person in the room who can tell good code from code that merely passes the tests."



Elena

QA Engineer - 8 years

— *professionally suspicious* —

"My job is finding what breaks. I'm going to need edge cases. Lots of them."



Rafael

Product Manager - 10 years

— *smoothly disengaged* —

"I'm already mapping this to an OKR. Can we circle back to the 'how' after?"



Jessica

Junior Dev - 1 year

— *quietly terrified* —

"I'm nodding along but internally I'm Googling half the words. Is that normal? Please say yes."

 : Monday, 9:00 AM

”

A contractor bailed halfway through a generic fan site. Pick your favorite show and turn this into your fan hub — you have until end of day, and you're all using GitHub Copilot.

”



Sarah needs proof

Fifteen years of hype cycles.
She'll commit when the output
speaks for itself — not before.



Marcus wants to ship

He builds infrastructure, not apps.
Today he'll use Copilot to go
further than he normally could
alone.



David fears generic output

Senior engineer, high standards.
His concern: AI that doesn't know
their patterns will make things
worse.

The answer to all three concerns is the same: configuration.

Modules 1–6 show exactly how to teach Copilot your team, your codebase, and your standards.

Build every layer of a Copilot-native workflow — on a real, running app

Eight hours. Six modules. Skills you'll use tomorrow.

✓ By End of Day

- ✓ Copilot fully briefed on your codebase
- ✓ App converted to your team's chosen show
- ✓ Reusable prompt files for common tasks
- ✓ Agent skills that validate data & build features
- ✓ Live context piped in via MCP servers
- ✓ Custom agents that plan and execute independently

Your Choice

Pick your favorite show:

-  The Office
-  Stranger Things
-  Breaking Bad
-  The Mandalorian
-  Game of Thrones
-  The Expanse

"Wait — eight hours? I thought this was a lunch thing."

— Marcus

BEFORE WE BUILD ANYTHING

Three ideas worth carrying into everything else today

The workshop will teach you *how* to use GitHub Copilot. These slides are about *why it works the way it does* — and who you're becoming in the process.



The Evolution Arc

Where developers have been, where they're going, and where today fits in that arc.



The Mindset Shift

The single biggest change in how you think about your work — and why it unlocks everything else.



The 5 Principles

Five patterns that show up in every module. Learn them once — they compound all day.

Our Evolution



1990 - 2015

Syntax Wizards

- ▶ Memorize language quirks
- ▶ Clever, unreadable code
- ▶ Knowledge hoarding
- ▶ Fast typing above all

✗ Onboarding took months



2015 - 2020

YAML Cowboys

- ▶ Configuration as code
- ▶ Infrastructure everywhere
- ▶ Copy-paste engineering
- ▶ Tool proliferation

✗ YAML bugs broke deploys



2020 - Present

Markdown Whisperers

- ✓ Clear intent over syntax
- ✓ Understandable over clever
- ✓ Scaled knowledge via docs
- ✓ Fast thinking over fast typing

✓ Copilot handles syntax — you handle judgment

💡 **Markdown is the medium** — the best teams will be those with the best prose, not the most elegant syntax

Three ways yesterday's work means something different now.

None of them require starting over.

Memory → Judgment

Knowing the right API used to be your edge.

Knowing when Copilot's answer is *wrong* is your edge now. That takes everything you've built.

Typing Speed → Thinking Speed

The bottleneck used to be your hands.

The bottleneck moved upstream. The constraint now is how fast you can think clearly — and how precisely you can say what you mean.

Knowledge Hoarding → Knowledge Scaling

Your expertise used to live in your head.

Now it lives in your docs — and Copilot runs on it. Writing things down stopped being overhead. It became leverage.

"I thought being junior meant I had to prove I knew as much as David. Apparently I just have to think clearly and write things down. I can do that."

— Jessica

The 5 Principles

Five patterns that show up in every module today. Learn them once — you'll see them compound all day.

1

Clarity Over Cleverness

If I can't explain it plainly to a coworker, I can't prompt it effectively to Copilot.

2

Intent Over Implementation

Describe the outcome I want — not how to build it.

3

Documentation as Leverage

Write once, benefit infinitely — every doc is a force multiplier for humans and Copilot alike.

4

Dialogue, Not Delegation

Start rough, iterate, refine. Good results come from conversation, not one perfect prompt.

5

Validate, Don't Automate

Copilot proposes. I decide. Every output earns its place by surviving my judgement.

 Clarity Over Cleverness

Clear thinking enables effective Copilot collaboration. If I can't explain it plainly to a human, I can't prompt it effectively to AI. The quality of the output starts with the quality of my thinking.

 MOVE AWAY FROM

- "Elegant" complexity for its own sake
- Obscure variable names and patterns
- Code written to impress, not communicate

→ MOVE TOWARD

- Simple, named, readable prompts and code
- Explaining your intent before the solution
- Asking "can I explain this plainly?"

 MOVE AGAINST

- The instinct to show technical depth
- One-liners that "work" but can't be read
- Complexity as a proxy for competence

"I kept looking for the sophisticated trick. Turns out 'just say what you mean' was the whole answer. Nobody told me that was a skill."

— Jessica

Intent Over Implementation

Describe WHAT outcome I need, not HOW to build it. My expertise is knowing what to build and what constraints matter — not prescribing the architecture without understand the problem.

MOVE AWAY FROM

- Describing implementation steps in prompts
- Prescribing the solution before the goal
- Micromanaging the AI's approach

→ MOVE TOWARD

- Goals, constraints, and success criteria
- "Here's what done looks like" framing
- Letting Copilot choose the path to your goal

MOVE AGAINST

- Prescribing architecture before the problem
- Over-specifying when the goal is clear
- Treating Copilot like a compiler, not a collaborator

"I've been asking engineers to lead with outcomes for a decade. They kept giving me architecture diagrams. Now there's a principle about it. I feel vindicated and also tired."

— Rafael

 Documentation as Leverage

Write once, benefit infinitely. Good documentation sharpens Copilot's output and scales my team's knowledge simultaneously. Every doc I write is a force multiplier — for humans and for Copilot.

 MOVE AWAY FROM

- "I'll document it later"
- Tribal knowledge that lives in Slack
- Docs that are out of date by default

→ MOVE TOWARD

- Docs as a first-class workflow artifact
- Architecture, decisions, and patterns in files
- Writing for Copilot as well as teammates

 MOVE AGAINST

- Knowledge that only lives in someone's head
- Teams that can't onboard without the expert
- Context that disappears when people leave

"Every doc I don't write becomes a Slack thread I answer seventeen times. I just never did the math on that before."

— Marcus

🔄 Dialogue, Not Delegation

Copilot works best as a conversation partner, not a one-shot task runner. Start rough, iterate, refine. The instinct to write one perfect prompt is understandable — but good results come from dialogue, not dictation.

📺 MOVE AWAY FROM

- Writing one massive prompt and waiting
- Giving up when the first result is rough
- Treating every session as a fresh start

→ MOVE TOWARD

- Start rough, refine in conversation
- Build shared context across a session
- Talk to Copilot as a coworker, not a tool

⚡ MOVE AGAINST

- Treating Copilot as a one-shot task executor
- Blaming the model when iteration would fix it
- Expecting perfection without conversation

"I kept rewriting the prompt from scratch every time it was wrong. I thought asking again meant I didn't know what I was doing. Turns out that's just the method."

— Jessica

Validate, Don't Automate

Copilot proposes; I decide. My expertise catches what Copilot misses. The goal is amplification, not replacement — every output is a proposal that earns its place by surviving my judgement.

MOVE AWAY FROM

- Accepting output without critical review
- Shipping Copilot-generated code unread
- Assuming confidence = correctness

→ MOVE TOWARD

- Every suggestion earns its place
- Your domain expertise as the quality filter
- Review as a skill, not a bottleneck

MOVE AGAINST

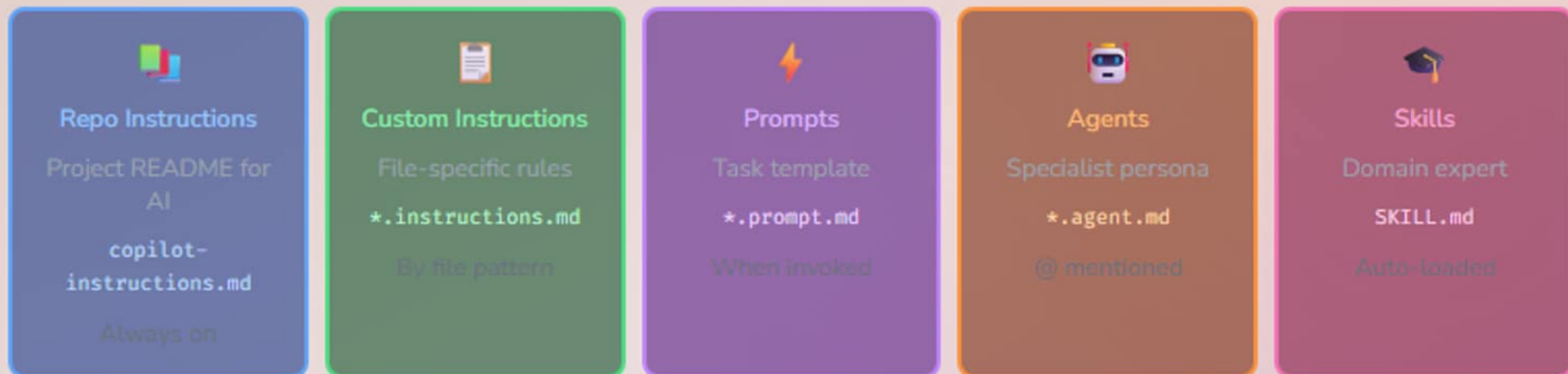
- Automation that removes human judgment
- Speed as a reason to skip review
- Delegating architectural decisions to AI

"I don't accept output I haven't validated. I never have — for developers, for pipelines, or for AI. This isn't a special rule I invented for Copilot. It's just the rule."

— Elena

The Customization Map

Five ways to customize GitHub Copilot—learn when to use what



"I need Copilot to understand our conventions and constraints..."

 Repo Instructions

"My test files need different rules than my source..."

 Custom Instructions

"We do this same task every sprint..."

 Prompts

"I want a reviewer that only knows our patterns..."

 Agents

"Copilot keeps guessing at our domain model..."

 Skills

Exercise Roadmap

#	Exercise	Lead	Time	What You'll Do
0.1	Clone FanHub & Pick Your Stack	All ★	10 min	Clone MSBart2/FanHub , pick your language flavor, get the app running locally
0.2	Experience "The Struggle"	All ★	15 min	Build with unintelligent Copilot —feel the pain
0.3	Master Context Variables	All ★	10 min	Learn <code>#file</code> , <code>@workspace</code> , <code>#codebase</code> , <code>#fetch</code> , <code>images</code>

💡 Exercise 0.2 is Critical

Copilot without any customizations will be frustrating, no matter the model we choose. Here's our chance to feel the pain of someone who just *types a prompt and hopes for the best*. Suffering or laughing here will make the rest of the workshop make an impact.

✖ The "Before"

What You'll Experience

- ✖ Generic suggestions
- ✖ Wrong architectural patterns
- ✖ Show-generic content
- ✖ Inconsistent styles
- ✖ Multiple failed attempts

💡 Team Reactions

Marcus: "Fighting with autocomplete..."

Jessica: "Am I using it wrong?"

Sarah: "Generic garbage."

David: "Copilot doesn't know our patterns."

🌟 The Promise

After Configuration

- ✓ Context-aware suggestions
- ✓ Follows your architecture
- ✓ Show-specific content
- ✓ Consistent patterns
- ✓ First-try success

The Transformation

From frustration to flow.

From generic to domain-specific.

From guessing to understanding.

By the end of Module 0, we'll have...



Environment Ready

FairHub running locally



Frustration Felt

Experienced generic AI struggles



Show Selected

Your fair data training datasets



Mindset Shifted

5 principles internalized

Ready to see everything change in **Module 1**

The Compounding Effect

Each layer unlocks the next — and makes every previous layer smarter



Module 1: Instructions → Project-wide context & standards

foundation



Module 2: Agent Plan Mode → Structured thinking before coding

+reasoning



Module 3: Custom Prompts → Reusable, context-aware templates

+recall



Module 4: Agent Skills → Domain-specific capabilities

+expertise



Module 5: MCP Servers → Live data & external context

+livedata



Module 6: Custom Agents → Specialist AI teammates

✦ full power

4 exercises · ~40 minutes · everyone leads

0.1 — Clone FanHub

Sarah ★

Get the app running locally. Explore the codebase before Copilot knows anything about it.

Establish your baseline → know what you're working with

0.2 — The Struggle

Jessica ★

Use Copilot cold — no instructions, no context setup. Notice what it gets wrong and why.

Feel the problem before you solve it

0.3 — Context Fundamentals

Elena ★

Explore what Copilot actually sees. Learn the difference between what you think it knows and what it knows.

Context isn't assumed — it's constructed

0.4 — Success Metrics

Rafael ★

Define what "better" looks like for your team. Set a baseline you can measure against at the end of the day.

if you can't measure it, you can't know it worked

"I asked Copilot to explain the codebase. It gave me a confident answer about files that didn't exist. That was exercise 0.2. It was extremely educational."

— Jessica

0.1

EXERCISE

Clone FanHub & Pick Your Stack

Everyone ★

🕒 10 min

Sarah drops the challenge. Before Copilot can help, it needs a codebase. **First things first: get the repo.**

📄 STEPS

- 1 `git clone https://github.com/MSBart2/FanHub.git`
- 2 Browse language folders, pick the one that matches your background
- 3 Pick your show — 🏢 The Office · 🧛 Stranger Things · 🚬 Breaking Bad · 🛡️ Mandalorian
- 4 Install deps, start frontend/backend (2 terminals), confirm the starter UI loads
- 5 Open in VS Code — confirm GitHub Copilot is active in the status bar

✅ SUCCESS CRITERIA

☐ Clone repo successfully☐ Language stack chosen☐ App running locally☐ VS Code - Copilot active

💡 *Jessica: "I don't need to know the whole codebase — just enough to start asking good questions."*

0.2

EXERCISE

Experience "The Struggle"

Everyone ★

🕒 15 min

⚠️ This exercise is intentionally frustrating. It helps to feel the "before" state to appreciate the transformation. Do not skip it.

📄 STEPS

- 1 Browse the FanHub code — schema, routes, components. Find the docs folder. It's empty.
- 2 Copilot, add a character detail page to FanHub
- 3 Note every frustration: wrong language, patterns, generic names, inconsistent style, multiple failed attempts

✗ YOU'LL EXPERIENCE

Generic suggestions that don't match your architecture

Confusingly incorrect patterns/syntax

Inconsistent code styles (because the practice is inconsistent)

Multiple failed attempts to get what you actually want

🗣️ Sarah: "Generic garbage. This is exactly what I expected."

0.3

EXERCISE

Context Fundamentals

Everyone



10 min

Marcus asks Copilot a basic question. It responds with a generic answer because it cannot see the project file he actually means. **Context is everything.**

TRY EACH MECHANISM

KNOWN FILE CONTEXT

Explain how this project is structured.
#file:README.md

PROJECT VISIBLE

@workspace how is this application organized?

#codebase where does this project handle routing?

SUCCESS CRITERIA

- ☐ Responds to each of the 3 prompts above
- ☐ Little or difference in response quality
- ☐ Prompts use context (file/path/workspace)

Marcus: "I asked without context, it answered without context, and I was annoyed. Then I added @workspace. Completely different answer. I was the problem. This time."

0.4

EXERCISE

Set Your Success Metrics

Everyone ★

🕒 10 min

David stops mid-conversation. *"Hold on — we haven't defined what success looks like. I'm not spending eight hours without a measurable outcome."* **He's not wrong.**

 STEPS

- 1 Scan the Old → New metrics. Pick 2–3 that feel true about how you measure yourself today.
- 2 For each one, ask honestly: *"Is this still the right thing to measure?"*
- 3 Pick one you want to be different by the end of module 6.
- 4 Hold it in mind — no need to write it down. You'll know if it shifted.

 OLD → NEW METRICS

Lines of code / day

Features shipped / week

Memorized API signatures

Clarity of intent in prompts

Being the only expert

Knowledge scaled to the team

Fast typing

Fast design thinking

💡 **David:** *"My 20 years of experience isn't obsolete — it's the quality filter."*

The Team, After Orientation

Jessica

The Eager Junior

"I thought this was going to be different. It answered questions about a completely different codebase. I'm nodding along but I'm genuinely confused about what just happened."

Still nodding. Not sure why.

David

The Seasoned Architect

"It produced three incorrect file paths with complete confidence. I have significant questions about the epistemology of this system."

The questions are not rhetorical.

Sarah

The Skeptical Senior

"I've seen tools that don't work. This is a tool that doesn't work yet. I'm not sure that's a better problem."

Holding judgment. Barely.

Elena

The Quality Champion

"It invented an API endpoint. Confidently. Without hesitation. I'm going to need a minute before I continue."

The minute is not going well.

Marcus

The DevOps Developer

"It gave me completely wrong information and seemed fine about it. I don't know if that's a me problem or an it problem. I'm not happy either way."

Back to Slack. Temporarily.

Rafael

The Product Visionary

"I asked a simple question. I got a confident answer about a product that doesn't exist. Someone is going to need to explain the ROI on 'confident but wrong.'"

ROI calculation: pending.

COMING UP NEXT

Module 1: Instructions

Sarah and David establish the foundation that makes everything else possible

🕒 10:00 AM

📄 copilot-instructions.md

🌟 Same code, better AI

M1
Instructions

M2
Plan Mode

M3

M4

M5

M6

⚠️ Don't Skip Exercise 0.2

The transformation only matters if you experience the problem first

🏠 YOU'LL BUILD

- ARCHITECTURE.md — project context for AI
- .github/copilot-instructions.md
- Path-scoped .instructions.md files

🌟 YOU'LL SEE

- Copilot go from generic to project-aware
- PR corrections start to disappear
- The baseline that makes every later module smarter



The 8-Hour Challenge Begins

 Module 0: Orientation — Complete



Pick Your Show

Choose your fan hub theme



Feel the Struggle

Exercise 0.2 — don't skip it



Then Transform

Module 1 changes everything